

Enhancer

Full Stack Developer Assignment

Angular & .NET Core — Developer Implementation Guide

Purpose of this guide

This document breaks the Enhancer assignment into a concrete, buildable plan: what each screen must do, the exact request/response shapes, the calculation rules, the recommended project structure, and a step-by-step build order. Use it as your working reference while implementing the solution, and as prep material for the technical interview.

Assignment source: Enhancer – Full Stack Developer (Angular & .NET Core) Assignment PDF, provided by careers@enhancer.com.

Stack: Angular (latest) + TypeScript · ASP.NET Core Web API (.NET 8 preferred) + C# · SQL Server.

Table of Contents

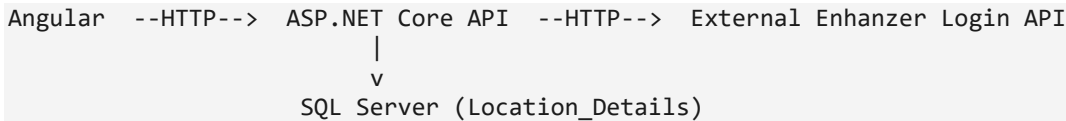
Table of Contents	2
1. Assignment Overview	3
1.1 Non-negotiable architecture rule	3
1.2 Technology checklist	3
2. Task 1 — Login Page	3
2.1 UI requirements	3
2.2 External login API contract	3
2.3 Login logic checklist (from the PDF, verbatim requirements)	4
2.4 Suggested validation messages	4
2.5 What your backend does on POST /api/auth/login	5
2.6 Session strategy	5
3. SQL Server Database	5
4. Task 2 — Purchase Bill Page	6
4.1 Route protection	6
4.2 Form fields	6
4.3 Calculation rules (exact formulas from the PDF)	6
4.4 “Add” button behaviour	7
Example	7
5. Recommended Backend Structure (ASP.NET Core)	7
6. Recommended Frontend Structure (Angular)	8
7. API Endpoints (your own design)	8
8. Cross-Cutting Requirements	8
8.1 Loading indicators	9
8.2 Error handling	9
8.3 Responsive UI	9
9. Recommended Build Order	9
10. Technical Interview Preparation	10
10.1 Angular concepts to be ready to explain	10
10.2 ASP.NET Core concepts to be ready to explain	10
10.3 SQL Server concepts to be ready to explain	10
10.4 Software engineering concepts to be ready to explain	10
11. Final Submission Checklist	11
11.1 Minimal README outline	11

1. Assignment Overview

The assignment asks for a small two-page web application:

- Page 1 — Login, authenticating against an external Enhancer API.
- Page 2 — Purchase Bill form, only reachable after a successful login.

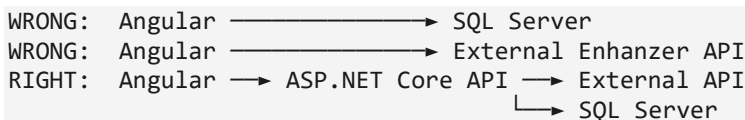
What is actually being evaluated is not the UI polish. It is whether you can wire together a realistic multi-tier flow correctly and explain every part of it:



1.1 Non-negotiable architecture rule

Never call the database or the external API directly from Angular

Angular only ever talks to your own ASP.NET Core API. Your API is the only thing that talks to the external Enhancer endpoint and to SQL Server. This separation is one of the first things an interviewer will check.



1.2 Technology checklist

Layer	Technology
Frontend	Angular (latest), TypeScript, Reactive Forms
Backend	ASP.NET Core Web API, .NET 8, C#
Database	SQL Server (script must be submitted as .sql)
External dependency	Enhanser staging login API (POS_Api/Invoke)

2. Task 1 — Login Page

2.1 UI requirements

A single simple form, matching the provided screenshot:

- Heading: “Hi”
- Email field (required)
- Password field (required, with show/hide toggle icon)
- A “LOGIN” button

Build this as a standalone LoginComponent using Angular Reactive Forms (FormGroup / FormControl), not template-driven forms — this is easier to unit test and to explain.

2.2 External login API contract

Endpoint (called from your backend, never directly from Angular):

```
POST https://ez-staging-api.azurewebsites.net/api/External_Api/POS_Api/Invoke
```

Request body — Company_Code and Username both take the value typed into the Email field; Pw takes the value typed into the Password field:

```
{
  "API_Action": "GetLoginData",
  "Device_Id": "D001",
  "Sync_Time": "",
  "Company_Code": "<email>",
  "API_Body": {
    "Username": "<email>",
    "Pw": "<password>"
  }
}
```

On success, the response includes a User_Locations array:

```
"User_Locations": [
  { "Location_Code": "LOC001", "Location_Name": "Colombo" },
  { "Location_Code": "LOC002", "Location_Name": "Chilaw" }
]
```

Design C# models for both the request and the response so the payload is strongly typed end-to-end — this is one of the “strong typing” points the assignment explicitly asks for.

```
public class LoginRequestDto
{
    public string Email { get; set; }
    public string Password { get; set; }
}

public class ExternalLoginResponse
{
    public bool Success { get; set; }
    public string Message { get; set; }
    public List<LocationDto> User_Locations { get; set; }
}

public class LocationDto
{
    public string Location_Code { get; set; }
    public string Location_Name { get; set; }
}
```

2.3 Login logic checklist (from the PDF, verbatim requirements)

- Validate user input (required fields, valid email format).
- Prevent access if authentication fails.
- Display meaningful validation and error messages.
- Securely store the authenticated session.
- Protect the Purchase Bill page from unauthorized access.
- On success, persist User_Locations to SQL Server, table Location_Details.

2.4 Suggested validation messages

Condition	Message shown to user
Email left empty	Email is required
Email not a valid format	Please enter a valid email
Password left empty	Password is required
External API rejects credentials	Invalid username or password
External API unreachable / 5xx	Unable to authenticate. Please try again.

2.5 What your backend does on POST /api/auth/login

1. Receive { email, password } from Angular
2. Build the external API request (Company_Code/Username = email, Pw = password)
3. Call the external Enhancer API with HttpClient
4. If authentication fails -> return 401 with a clean error message
5. If authentication succeeds:
 - a. Save/replace rows in Location_Details from User_Locations
 - b. Issue a session token (JWT or secure cookie) back to Angular
6. Angular stores the token/session and redirects to /purchase-bill

2.6 Session strategy

The assignment says only “securely store the authenticated session” — it does not mandate JWT. A JWT issued by your own API (separate from whatever the external API returns) is a safe, easy-to-explain default:

- Backend issues a short-lived JWT after a successful external login.
- Angular stores it (e.g. in a signal/service-held variable backed by sessionStorage) and attaches it via an HttpInterceptor as an Authorization: Bearer header.
- An AuthGuard on the /purchase-bill route checks that a valid token exists before allowing navigation.
- Document whichever mechanism you pick in the README, since the PDF explicitly leaves this open.

3. SQL Server Database

Create a dedicated database, e.g. EnhancerAssignmentDB, with a Location_Details table populated from User_Locations after every successful login.

```
CREATE DATABASE EnhancerAssignmentDB;
GO
USE EnhancerAssignmentDB;
GO

CREATE TABLE Location_Details
(
    Id                INT IDENTITY(1,1) PRIMARY KEY,
    Location_Code     NVARCHAR(100) NOT NULL,
    Location_Name     NVARCHAR(200) NOT NULL,
    CreatedAt         DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);
```

Decide (and be ready to explain) whether each login appends new rows or clears and re-inserts the current user's locations — either is acceptable, but pick one intentionally rather than by accident. A common, defensible approach: delete existing rows for that company/user, then bulk-insert the fresh User_Locations on every login.

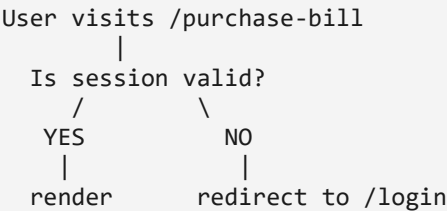
Access this table only through Entity Framework Core (DbContext + a LocationRepository), never with raw ADO.NET calls scattered through controllers — this is what “clean architecture” and “separation of concerns” are checking for.

4. Task 2 — Purchase Bill Page

4.1 Route protection

The route /purchase-bill must be unreachable without a valid session:

```
{ path: 'purchase-bill', component: PurchaseBillComponent, canActivate: [authGuard] }
```



4.2 Form fields

Field	Behaviour
Item	Autocomplete text input. Allowed values: Mango, Apple, Banana, Orange, Grapes, Kiwi, Strawberry.
Batch	Dropdown populated from Location_Name in Location_Details (fetched via your own API — never hardcoded).
Standard Cost	Numeric input.
Standard Price	Numeric input.
Quantity	Numeric input, must be greater than 0.
Discount	Percentage input (0–100).
Total Cost	Calculated, read-only.
Total Selling	Calculated, read-only.

4.3 Calculation rules (exact formulas from the PDF)

```
Total Cost      = (Standard Cost × Quantity) - Discount%
Total Selling    = Standard Price × Quantity
```

Worked example given in the PDF — Standard Cost 100, Standard Price 150, Quantity 5, Discount 20%:

```
Total Cost      = (100 × 5) - 20% of 500 = 500 - 100 = 400
Total Selling    = 150 × 5                = 750
```

Implement this once as a pure function (e.g. in a shared calculation service or utility) and reuse it for both the live preview and the row added to the table, so the two can never drift apart.

4.4 “Add” button behaviour

- Validate the row (Item and Batch selected, Quantity > 0, Discount between 0 and 100) before adding.
- Append the row to the items table shown below the form.
- Recalculate the Item Summary panel immediately.

Item Summary rules:

Total Items = number of rows currently in the table
Total Quantity = sum of the Qty column across all rows

Example

Item	Batch	Qty
Mango	Colombo	5
Apple	Chilaw	3
Banana	Kandy	2

Total Items = 3, Total Quantity = 5 + 3 + 2 = 10.

5. Recommended Backend Structure (ASP.NET Core)

```
Backend/
├── Controllers/
│   ├── AuthController.cs
│   ├── LocationController.cs
│   └── PurchaseBillController.cs
├── Models/
│   ├── Location.cs
│   └── PurchaseBill.cs
├── DTOs/
│   ├── LoginRequestDto.cs
│   ├── LoginResponseDto.cs
│   └── LocationDto.cs
├── Services/
│   ├── AuthService.cs          (calls external API, issues session)
│   ├── LocationService.cs
│   └── PurchaseBillService.cs
├── Repositories/
│   └── LocationRepository.cs
├── Data/
│   └── ApplicationDbContext.cs
└── Program.cs
```

Keep a strict one-way dependency flow: Controller → Service → Repository → Database. Controllers should contain almost no logic beyond model binding, calling a service, and mapping the result to an HTTP response.

Register CORS explicitly in Program.cs so the Angular dev server (e.g. <http://localhost:4200>) is allowed to call the API, use dependency injection for every service/repository, and wrap external calls and DB calls in try/catch with a consistent error-response shape (e.g. { message, statusCode }).

6. Recommended Frontend Structure (Angular)

```
src/app/
├── core/
│   ├── guards/
│   │   └── auth.guard.ts
│   ├── interceptors/
│   │   └── auth.interceptor.ts
│   └── services/
│       ├── auth.service.ts
│       └── api.service.ts
├── features/
│   ├── login/
│   │   ├── login.component.ts
│   │   ├── login.component.html
│   │   └── login.component.css
│   └── purchase-bill/
│       ├── purchase-bill.component.ts
│       ├── purchase-bill.component.html
│       └── purchase-bill.component.css
├── shared/
│   ├── components/    (e.g. a reusable autocomplete / item-table component)
│   └── models/        (TypeScript interfaces mirroring the backend DTOs)
└── app.routes.ts
```

Give every API shape a TypeScript interface (LoginRequest, LoginResponse, Location, PurchaseBillItem) instead of using any — this is the “strong typing” requirement. Pull the items table and the item-summary panel out into small reusable components so “reusable components” is demonstrably true, not just claimed in the README.

7. API Endpoints (your own design)

The PDF does not prescribe internal endpoint names, so design them yourself and document them. A reasonable set:

Method & Path	Purpose
POST /api/auth/login	Validates credentials against the external API, saves locations, returns a session token.
GET /api/locations	Returns saved Location_Name values for the Batch dropdown.
POST /api/purchase-bills	Accepts a new purchase-bill line item (optional, if you persist bill items server-side).

8. Cross-Cutting Requirements

8.1 Loading indicators

Show feedback during every network call: a spinner or a disabled button with “Loading...” text while logging in, and a “Loading locations...” state while the Batch dropdown is being populated.

8.2 Error handling

Failure	User-facing message
External login API fails	Unable to authenticate. Please try again.
Database read/write fails	Unable to load locations.
Invalid form input	Quantity must be greater than 0. (etc.)

Never surface a raw exception message or stack trace to the user; log details server-side and return a clean, generic message.

8.3 Responsive UI

The Purchase Bill layout is form + summary side-by-side on desktop; stack them vertically (form, then summary, then table) on narrower viewports using CSS flexbox/grid with a breakpoint.

9. Recommended Build Order

Do not write the whole thing in one pass. Build and verify in this order:

1. Create the Angular project.
2. Create the ASP.NET Core (.NET 8) Web API project.
3. Set up SQL Server and create the database.
4. Create the Location_Details table.
5. Build the Login UI (Reactive Form + validation messages).
6. Implement POST /api/auth/login on the backend.
7. Connect the backend to the external Enhancer API.
8. Parse and return User_Locations in the login response.
9. Persist locations to SQL Server via EF Core.
10. Implement session issuance (JWT or equivalent) and secure storage on the client.
11. Implement the Angular AuthGuard and HttpInterceptor.
12. Build the Purchase Bill form UI.
13. Load Location_Name values into the Batch dropdown from GET /api/locations.
14. Implement the Item autocomplete.
15. Implement the Total Cost / Total Selling calculations.
16. Implement Add → append row to the items table.
17. Implement the Item Summary (Total Items / Total Quantity).
18. Add error handling and loading indicators throughout.
19. Make the layout responsive.

20. Write the SQL script, README, push meaningful commits, and record the demo video.

10. Technical Interview Preparation

AI-assisted development is allowed — but you must be able to explain it

The PDF explicitly permits using ChatGPT / Copilot / Claude, but requires that you fully understand your own implementation and can explain any part of it in the interview. Don't submit code you can't walk through line by line.

10.1 Angular concepts to be ready to explain

- Components & component lifecycle
- Reactive Forms and validators
- Services and dependency injection
- HttpClient and Observables (subscribe, pipe, error handling)
- Routing and route guards
- HTTP interceptors
- TypeScript interfaces / strong typing
- Loading and error state management

10.2 ASP.NET Core concepts to be ready to explain

- Controllers and routing
- Dependency injection
- DTOs vs domain models
- Services and repositories
- Entity Framework Core and DbContext
- HttpClient usage for calling the external API
- Exception handling middleware
- Configuration (appsettings.json, connection strings)
- CORS configuration

10.3 SQL Server concepts to be ready to explain

- Database and table design
- Primary keys
- INSERT / SELECT statements
- Connection strings
- EF Core migrations vs a hand-written SQL script

10.4 Software engineering concepts to be ready to explain

- Clean architecture / separation of concerns
- Why components/services are reusable in your solution

- Why you chose your particular session/auth mechanism
- Your Git commit history and what each commit represents

11. Final Submission Checklist

- GitHub repository URL, with meaningful, incremental commit messages (not a single “initial commit”).
- SQL Server database script (.sql) checked into the repo, e.g. database/EnhancerAssignment.sql.
- README.md with clear setup instructions for backend, frontend, and database.
- 5–10 minute screen recording demonstrating login, the locations being saved, and the full Purchase Bill flow.
- Completed submission form.

11.1 Minimal README outline

```
# Enhancer Assignment

## Technologies
Frontend: Angular, TypeScript
Backend: ASP.NET Core (.NET 8), C#
Database: SQL Server

## Setup
### Backend
dotnet restore
dotnet run

### Frontend
npm install
ng serve

## Database
Execute database/EnhancerAssignment.sql against your SQL Server instance,
then update the connection string in appsettings.json.

## Notes on session/auth approach
<explain whatever mechanism you implemented, e.g. JWT>
```

Bottom line

The core challenge is not the visual design — it's demonstrating a correct Angular → ASP.NET Core → External API → SQL Server → Angular round trip, with real validation, authentication, clean layering, and code you can defend in the interview.